

手摸手之乘车人模块开发

乘车人表结构

以下表结构为乘车人原始表结构，而大家执行过建表语句后，看到数据库中大多为 t_passenger_0-15 的表名。这是因为，考虑到乘车人数据量过大，对乘车人表进行了分库分表处理。

```
CREATE TABLE `t_passenger` (  
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT 'ID',  
  `username` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '用户名',  
  `real_name` varchar(128) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '真实姓名',  
  `id_type` int(3) DEFAULT NULL COMMENT '证件类型',  
  `id_card` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '证件号码',  
  `discount_type` int(3) DEFAULT NULL COMMENT '优惠类型',  
  `phone` varchar(128) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '手机号',  
  `create_date` datetime DEFAULT NULL COMMENT '添加日期',  
  `verify_status` int(3) DEFAULT NULL COMMENT '审核状态',  
  `create_time` datetime DEFAULT NULL COMMENT '创建时间',  
  `update_time` datetime DEFAULT NULL COMMENT '修改时间',  
  `del_flag` tinyint(1) DEFAULT NULL COMMENT '删除标识',  
  PRIMARY KEY (`id`),  
  KEY `idx_id_card` (`id_card`) USING BTREE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='乘车人表';
```

分库分表策略

乘车人数据分库分表相关文档参考下述文档。

[乘车人数据如何实现分库分表](#)

敏感数据脱敏展示

乘车人敏感信息，比如身份证、手机号等数据查询脱敏展示相关参考下述文档。

技术亮点

本章节通过乘车人查询和修改接口给大家带来 HTTP 接口防重复提交、SpEL以及缓存与数据一致性等核心技术点。

直连数据库

根据用户名查询乘车人数据接口。

```
// Controller
@GetMapping("/api/user-service/passenger/query")
public Result<List<PassengerRespDTO>> listPassengerQueryByUsername() {
    return
Results.success(passengerService.listPassengerQueryByUsername(UserContext.getUsername()))
;
}

// Service Impl
@Override
public List<PassengerRespDTO> listPassengerQueryByUsername(String username) {
    LambdaQueryWrapper<PassengerDO> queryWrapper =
Wrappers.lambdaQuery(PassengerDO.class)
        .eq(PassengerDO::getUsername, username);
    List<PassengerDO> passengerDOList = passengerMapper.selectList(queryWrapper);
    return BeanUtil.convert(passengerDOList, PassengerRespDTO.class);
}
```

修改乘车人相关信息接口。

```
// Controller
@PostMapping("/api/user-service/passenger/update")
public Result<Void> updatePassenger(@RequestBody PassengerReqDTO requestParam) {
    passengerService.updatePassenger(requestParam);
    return Results.success();
}

// Service Impl
```

```
@Override
public void updatePassenger(PassengerReqDTO requestParam) {
    PassengerDO passengerDO = BeanUtil.convert(requestParam, PassengerDO.class);
    LambdaUpdateWrapper<PassengerDO> updateWrapper =
Wrappers.lambdaUpdate(PassengerDO.class)
        // TODO 用户名和当前用户比对
        .eq(PassengerDO::getUsername, requestParam.getUsername())
        .eq(PassengerDO::getId, requestParam.getId());
    passengerMapper.update(passengerDO, updateWrapper);
}
```

HTTP防重复提交

假设，我在新增乘车人时，快速且连续点击了两次或者多次，数据库中可能会出现多条一模一样的数据。这就是重复提交问题。

什么是防重复提交

HTTP 接口防重复提交是一种机制，用于防止在某个请求还未完成处理或响应返回之前，重复提交相同的请求。防重复提交的目的是确保每个请求只被处理一次，避免因重复提交而引起的数据重复处理、重复写入等问题。

如果不做防重复提交，可能会导致以下问题：

1. 数据重复处理：如果用户在某个操作还在处理中时重复提交请求，可能会导致相同的数据被处理多次，造成数据的重复操作和处理逻辑的错误。
2. 重复写入：在某些场景下，请求可能触发对数据库或其他存储系统的写入操作。如果请求被重复提交，可能会导致相同的数据被重复写入，破坏数据的一致性。
3. 资源浪费：重复提交请求可能会导致服务器资源的浪费。如果请求处理的时间较长，重复提交会占用服务器的处理能力，增加服务器的负载，降低系统的性能和吞吐量。
4. 业务逻辑错误：在某些业务场景下，重复提交可能会导致业务逻辑错误。例如，如果用户在生成订单的过程中重复提交订单请求，可能会导致生成多个相同的订单，引发订单的混乱和错误。

这种问题，可以通过前端来一定程度上避免，比如发出一次请求后，请求没有响应就不让再点击。类似于这种，如下图所示：

请核对以下信息

×

2023-09-27（周三）G35次北京南站（09:56开）——杭州东站（04:30到）

序号	席别	票种	姓名	证件类型	证件号码
1	商务座		方南	中国居民身份证	
2	商务座	成人票	8888	中国居民身份证	88888888

*如果本次列车剩余席位无法满足您的选座需求，系统将自动为您分配席位

🔊 选座咯

已选座0/2

窗

A

C

过道

F

窗

窗

A

C

过道

F

窗

本次列车，**商务座余票8,一等座余票138,二等座余票810,**

返回修改

确认

但是这只能一定程度上解决问题，因为如果绕过前端，直接请求后端接口，这个就起不到防护作用。

为了防止用户快速操作以及接口被恶意请求这种行为，需要针对购票接口做 HTTP 防重复提交处理。因为咱们针对 HTTP 的防重复提交和消息队列的幂等做了封装，所以直接引入组件库直接使用即可。

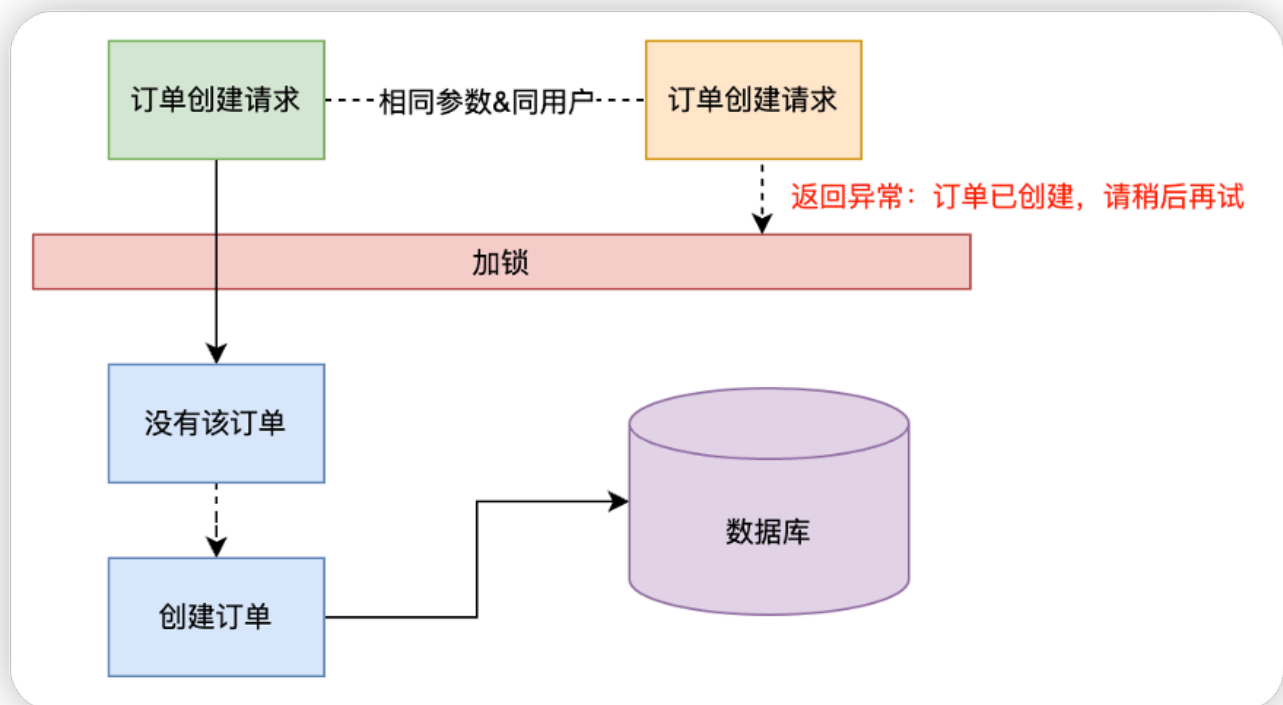
引入幂等组件

在购票服务中引入幂等组件 Maven Jar 坐标。

```
<dependency>
  <groupId>org.opengoofy.index12306</groupId>
  <artifactId>index12306-idempotent-spring-boot-starter</artifactId>
  <!-- 因为购票服务和幂等组件库使用的都是一个版本，直接取购票服务版本号即可 -->
  <version>${project.version}</version>
</dependency>
```

接口添加防重复提交注解

添加防重复提交注解之前，思考一个问题，防重复提交的运行原理是什么？是不是加一把锁就能解决了，运行原理如下所示：



我们在控制层新增乘车人接口上添加组件中的防重复提交注解。

```
@Idempotent(
    uniqueKeyPrefix = "index12306-user:lock_passenger-alter:",
    key =
        "T(org.opengoofy.index12306.frameworks.starter.user.core.UserContext).getUsername()",
    type = IdempotentTypeEnum.SPEL,
    scene = IdempotentSceneEnum.RESTAPI,
    message = "正在新增乘车人，请稍后再试..."
```

```
)
@PostMapping("/api/user-service/passenger/save")
public Result<Void> savePassenger(@RequestBody PassengerReqDTO requestParam) {
    passengerService.savePassenger(requestParam);
    return Results.success();
}
```

简单描述下这个注解的一些参数：

- scene：最开始和大家说过，组件库封了两种方案，HTTP 和消息队列，如果是 HTTP 选择使用 RESTAPI。
- type：HTTP 防重复提交有多种实现方案，如何构建唯一标识呢？SPEL 就是通过 key 参数运行时生成。
- key：填写 SpEL 表达式，运行时通过填写的 SpEL 表达式构建唯一的锁标识。
- uniqueKeyPrefix：区分业务的前缀，拼接上 key 参数用来生成真正唯一的锁标识。可不指定。
- message：满足防重复提交条件后，开始触发抛异常行为，异常中的提示信息。可不指定。

什么是 SpEL 表达式？

SpEL (Spring Expression Language) 是 Spring 框架提供的一种表达式语言，用于在运行时评估表达式。它支持在 Spring 应用程序中进行动态求值和访问对象的属性、方法调用、运算符操作等。

SpEL 表达式的语法类似于其他编程语言的表达式语言，具有以下特点：

1. 属性访问：可以使用点号 (.) 来访问对象的属性，例如 person.name。
2. 方法调用：可以通过在表达式中使用方法名和参数来调用对象的方法，例如 person.getName()。
3. 运算符操作：支持常见的算术运算符（如加减乘除）、逻辑运算符（如与、或、非）和比较运算符（如等于、大于、小于等）。
4. 条件表达式：支持条件表达式，例如三元运算符 condition ? value1 : value2。
5.

SpEL 功能很多，在这里就不一一举例说明了。接下来给大家解析下咱们这个接口的 SpEL 表达式是什么？

```
T(org.opengoofy.index12306.frameworks.starter.user.core.UserContext).getUsername()
```

简单来说，通过 UserContext 容器获取当前登录用户名。

缓存一致性

从缓存读取乘车人数据

通过直连数据库版本得知，咱们根据用户名查询乘车人都是全部直接调用数据库的。这种模式在高并发海量流量的场景下，数据库会秒挂。

为此，我们需要通过缓存来防止请求直接打到数据库。封装的缓存组件 `safeGet` 的作用就是，如果缓存中有，那么就从缓存中返回，如果缓存中没有，就查询数据库，并将查询数据库的结果，同步到缓存。

会涉及到一个缓存击穿的问题，底层原理具体可以看我们这篇文章：[缓存击穿之双重判定锁如何优化性能？](#)

```
@Override
public List<PassengerRespDTO> listPassengerQueryByUsername(String username) {
    String actualUserPassengerListStr = getActualUserPassengerListStr(username);
    return Optional.ofNullable(actualUserPassengerListStr)
        .map(each -> JSON.parseArray(each, PassengerDO.class))
        .map(each -> BeanUtil.convert(each, PassengerRespDTO.class))
        .orElse(null);
}

private String getActualUserPassengerListStr(String username) {
    return distributedCache.safeGet(
        USER_PASSENGER_LIST + username,
        String.class,
        () -> {
            LambdaQueryWrapper<PassengerDO> queryWrapper =
Wrappers.lambdaQuery(PassengerDO.class)
                .eq(PassengerDO::getUsername, username);
            List<PassengerDO> passengerDOList =
passengerMapper.selectList(queryWrapper);
            return CollUtil.isEmpty(passengerDOList) ?
JSON.toJSONString(passengerDOList) : null;
        }
    );
}
```

```
    },  
    1,  
    TimeUnit.DAYS  
);  
}
```

思考一个问题，我们在抢票前，一般是不会查询乘车人数据的。拿着就会造成一个问题，抢票时会有大量请求去读读取数据库获取乘车人信息，进而造成数据库压力增加。这种问题如何解决？

是否能够在节假日前，把所有用户的乘车人数据加载到缓存？

不行。因为这种数据信息会非常大，对 Redis 内存的压力会变大，并且这些数据不够精准，因为大部分不会进行购票使用。

我想到一个方案，那就是当用户登录 12306 系统时，可以执行一个异步任务，让这个异步任务去读取数据库的乘车人数据并放入缓存。这样的话，既不影响登录操作，又可以减轻高并发情况下数据库的负载，完美。

乘车人数据变更了怎么办

既然我们已经把乘车人数据放到了缓存里，而且用户还能修改乘车人的数据信息。那就不得不面对一个问题，缓存和数据库的数据一致性如何保证？

因为我们已经写了比较详细的缓存与数据库数据一致性文档，这里就不再赘述。直接说解决方案：先写数据库，后删除缓存。

具体缓存与数据库一致性方案查看文档：[缓存与数据库一致性如何解决？](https://www.yuque.com/magestack/12306/hv1o38ac4c54t2rw)。

更新: 2023-09-27 16:30:34

原文: <<https://www.yuque.com/magestack/12306/hv1o38ac4c54t2rw>>